2023 International Conference on Modeling & E-Information Research, Artificial Learning and Digital Applications (ICMERALDA) | 979-8-3503-6935-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICMERALDA60125.2023.10458207

Recommending Public Transit Route: Ant-Colony Optimization or Dijkstra?

Abdul Jabbar
Informatics Engineering Department
Faculty of Computer Science
Universitas Brawijaya
Malang, Indonesia
aabdul.jabbar1301@gmail.com

Aryo Pinandito*
Information System Department
Faculty of Computer Science
Universitas Brawijaya
Malang, Indonesia
aryo@ub.ac.id

***Abstract*—Advances in algorithmic studies have been made recently, especially in the area of finding the shortest path. Despite taking a greedy approach, the Dijkstra algorithm is well recognized for its efficacy and has established itself as the industry standard for identifying the shortest path in a graph. Conversely, scientists have embraced the Ant-Colony Optimization (ACO) approach, which draws inspiration from the behavior of ants. This study examines the two methods and determines if, given equivalent resources, ACO can generate Dijkstra-like recommendations. Research data is subjected to both algorithms, and the outcomes are presented visually. In data measurement, Dijkstra is processed first to determine the best path, and then the ACO method is applied to produce findings that are similar to Dijkstra. In data measurement, Dijkstra is processed first to determine the best path, and then the ACO method is applied to produce findings that are similar to Dijkstra. Memory consumption, computation time, initial memory allocation, and final memory allocation are evaluation parameters. ACO is more efficient than Dijkstra in terms of execution time and number of pathways traversed, according to the study's repeated evaluation methodology. This iterative assessment procedure is carried out to address the possibility of single measurements resulting from the JavaScript Garbage Collector. The study's findings imply that a larger memory may be necessary for ACO to deliver the desired outcomes. These results could also be impacted by each algorithm's distinct qualities.**

***Index Terms*—algorithm, Ant-Colony Optimization, Dijkstra, shortest-path, transit, transportation**

I. Introduction

Dijkstra, Ant-Colony Optimization (ACO) [1], [2], and the A* algorithm [3] are algorithms used for determining the shortest path. One of the classic algorithmic problems is determining the shortest path between two points. The Dijkstra and Ant-Colony Optimization (ACO) techniques are widely used to find the shortest path. While ACO uses pheromones that affect the length of ant paths, Dijkstra has a high degree of accuracy in calculating the shortest distance between two nodes. By simulating an ant's search for the best path, the ACO method can be used to determine the shortest path. The Dijkstra technique, which uses a greedy strategy, can also be used as an alternative to determine the shortest path [4].

The determination of pathways, which entails the identification of the shortest, closest, or farthest paths, constitutes a classic algorithmic problem. However, this presents a problem that calls for an optimization viewpoint. This includes choosing the fastest path from point A to point B while taking into consideration restrictions like distance, physical characteristics, time cost, or other potential restrictions that may emerge in a particular setting. To find the shortest path and processing time, ACO and Dijkstra are frequently utilized. Dijkstra is a method with great accuracy but considerable computational overhead when working with huge datasets [5]–[8]. However, when working with huge datasets, the ACO method performs better in terms of CPU utilization [7].

Several studies have conducted investigations on the aspects of processing time, memory usage, and accuracy in the context of comparing the performance of Ant-Colony Optimization (ACO) and the Dijkstra algorithm [5]–[7]. The main goal of these studies has been to assess how well these approaches perform in different application scenarios. These research projects' conclusions have important ramifications for choosing the best algorithm for a given set of network conditions and problem domains. These studies mostly focused on variables like processing time (CPU time), memory usage, and the correctness of the solutions [7]. According to the findings of some of the research, Dijkstra may perform better in situations involving medium- to large-sized networks, while ACO may demonstrate higher efficiency in the context of large-scale networks. However, the size of the applied network as well as the unique requirements and features of the problem at hand should be taken into account while selecting between these two approaches.

This study aims to assess the degree to which route search problems can be solved using the Ant-Colony Optimization (ACO) approach and is comparable to those obtained with the Dijkstra method. Taking into account that the Dijkstra technique, despite the fact that it can be computationally time-consuming when working with large amounts of data, is renowned for its excellent accuracy and certainty because of its greedy approach. The goal of this study is to determine whether ACO can generate solutions that roughly have similar quality as those obtained by Dijkstra. The results of this study are expected to provide a more profound insight into the potential of Ant-Colony Optimization (ACO) in delivering

*Corresponding author.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on August 27,2026 at 09:44:20 UTC from IEEE Xplore. Restrictions apply.

results that are commensurate with Dijkstra, as well as an understanding of the factors that influence the relative performance of both methods.

II. Literature Review

A. Dijkstra Algorithm

The shortest-path problem is a key component of driver assistance and emergency route management systems. Dijkstra's algorithm can be used to discover the shortest path in order to solve such problems. In navigation systems, this is essential since it facilitates the making of quick and well-informed judgments. Consequently, the introduction of a new design known as the "shortest path" has made it possible for vehicles to find alternate routes swiftly and effectively during runtime. A study suggested that Dijkstra performs well in displaying the shortest path across a range of parameters and offers a comparable fast response time [9], However, because of its greedy approach, Dijkstra processes the data in a slow manner [4], [6].

Dijkstra is an algorithm used to determine the shortest path. Dijkstra algorithm for finding the shortest path in a weighted graph to find the shortest path between vertices [10]. Fig. 1 It is known that there are multiple other routes from point A to point I. The distance measurements between each point are unique. There are several different distances shown in the figure; the path that is closest is A, B, E, C, H, and I, which consist of 6 points. The nearest path is based on the distance between two points rather than the quantity of points. Dijkstra's functionality consists of the following steps:

- First, A will look at each and every result from the neighboring spots individually in the first phase. Dijkstra compares, for instance, points A to B, A to C, and A to D. These values are then recorded and compared to the points that haven't been traversed once more.
- Proceed to the next stop and repeat step 1 after detecting it. For instance, compare point B to point E and point B to point C, and so on, until you reach the goal point and there are no more points to be traversed.

The advantages of Dijkstra are its accuracy in determining the shortest distance between two nodes with a given weight; this results in an optimal solution for the shortest path problem. can be applied to weighted graphs with various types such as distance, cost, or travel time. This makes it useful in a variety of applications, including route finding. In addition, its disadvantages are its high time complexity, especially on very large graphs with lots of nodes, which results in it being very slow. Dijkstra also only applies to a positive weighted graph and also requires a large amount of memory when using a weighted graph with lots of nodes.

B. Ant-Colony Optimization

The purpose of this study is to increase package delivery efficiency for the post service. The findings indicate that while the shortest distance outcomes produced by both methods are comparable, Dijkstra's algorithm outperforms Ant Colony Optimization in terms of speed [8].

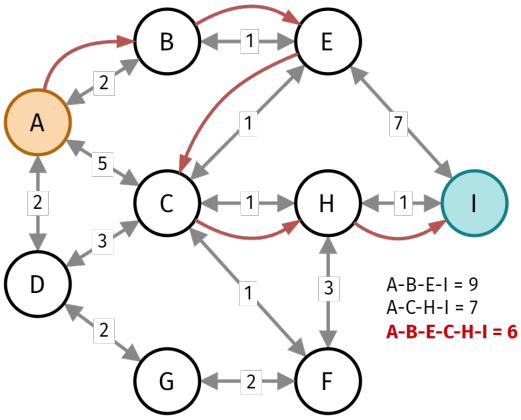


Fig. 1. Finding the shortest path using Dijkstra

The goal of this study is to give a solution for e-logistics service providers' vehicle route scheduling problem. Each vehicle in this challenge has to visit many pickup nodes, like warehouses, to pick up orders before delivering goods to the clients on the list. In real-world applications, the objective is to reduce the total cost of travel while taking into account a variety of constraints, including a variety of vehicles, capacity restrictions, time windows, the length of the driver's shift, and so forth. First, a mathematical model for the multiple pickup and multiple delivery vehicle routing problem with a time frame and heterogeneous fleets. Furthermore, this problem is solved on a wide scale using an ant colony optimization technique. [11].

In addition to the Dijkstra algorithm, there is Ant-Colony Optimization for determining the shortest distance. Ant-Colony Optimization (ACO) is an artificial intelligence algorithm developed based on the behavior of real ant colonies as they navigate from their nest to a food source [2], [12], [13]. ACO is used to solve combinatorial optimization problems by seeking the best solution through the collaboration of artificial ants that interact with each other (see Fig. 2). ACO's functionality consist of the following steps:

1) Initialization entails figuring out what each node's values are.
2) Path construction is the process by which each ant constructs a path using pheromones as a guide and other ant-specific rules.
3) After each ant has finished investigating a path, pheromones are added to the paths that the ants that have discovered successful solutions have traveled.
4) Pheromones gradually disappear over time to avoid convergence to less-than-ideal solutions.
5) To find better answers, carry out steps 2-4 repeatedly for a predetermined number of iterations.

C. Dijkstra and Ant-Colony Optimization in Solving Public Transit Recommendation Problem

The Dijkstra method is used in transportation to determine the shortest path between two points on a network or road

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on August 27,2026 at 09:44:20 UTC from IEEE Xplore. Restrictions apply.

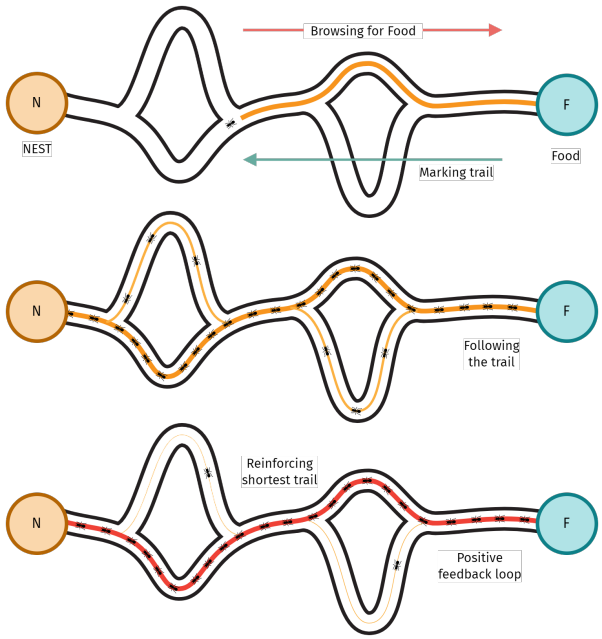


Fig. 2. Finding and reinforce the shortest path with Ant-Colony Optimization algorithm

map. The way the algorithm operates is by iteratively going over each point or node and figuring out the shortest paths between the source and the destination. Dijkstra does this by calculating the distance between each point in the network and the source and then methodically choosing the shortest way. Dijkstra offers an exact and ideal solution in the context of transportation in this way [14], [15].

The subject of school bus routing in metropolitan areas is the focus of this study. Because they involve drivers, passengers, schedules, and routes, transportation service issues are by their very nature complicated [16]–[18]. This study specifically addresses a variant of the vehicle routing problem, wherein three simultaneous decisions are made to minimize the total travel distance: choosing the set of stops to visit, allocating each student to the appropriate stop, and, finally, determining the routes to be taken with the selected stops. Ant Colony combined with the neighborhood local search algorithm produces consistently improved results, as demonstrated by the empirical data. [18].

ACO is an ant colony-based algorithm that uses the shortest path search principle of ant colonies [2], [12], [13], whereas Dijkstra is a Greedy algorithm that uses a node-based method. This is the primary technical distinction between ACO and Dijkstra [4], [6], [9]. The basic hypothesis is that these variations in methodology could have distinct effects on real-world application, which is why this study needs to investigate and address.

III. Methodology

The problem identification and identification procedure, which precedes the design process to address the researcher's set of questions, is where this research starts. Using public transportation line data and the website, an attempt was made to address the performance testing of approaches between Dijkstra and ACO (Ant-Colony Optimization). Within the parameters of the study, rendering or processing data will be gathered via experiments and examined. 1504 points from 6 transportation systems with inbound (1) and outgoing (5) conditions make up the data used. Figure 3 displays the research technique flowchart.

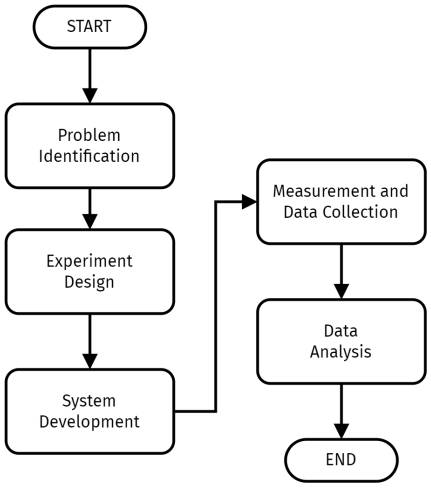


Fig. 3. Research Methodology

A. Problem Identification and Measurement

Formulating the Ant-Colony Optimization (ACO) method with the goal of producing the expected outcomes should yield a comparable result to the Dijkstra method in the same context. The goal of this research is to develop a strategy that enables the use of ACO to get expected results that are comparable to those of the Dijkstra method, which uses an exhaustive approach. This study might discover that route planning can be handled more effectively and efficiently using a non-exhaustive approach, such as ACO.

The Dijkstra method is used to create the ideal path before beginning the data measurement step. Dijkstra algorithm uses a greedy approach; thus, it is renowned for its high level of accuracy. In Dijkstra, every point or node is methodically examined. It carefully takes into account every possible shortest path from a source to a particular destination. By doing this, Dijkstra is able to determine the best path based on particular criteria, e.g., faster journey time and shorter distance. Therefore, the output of Dijkstra algorithm will be used as a guide for the ACO algorithm to decide when to end the search. When the results produced by the ACO are similar to or close to the result from Dijkstra, the search process of the ACO method will eventually end. This guarantees that ACO only uses the required resources more efficiently and yields a comparable result. Several evaluation parameters were collected, including memory usage and the computing times for each operation, as depicted in Fig. 4.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on August 27,2026 at 09:44:20 UTC from IEEE Xplore. Restrictions apply.

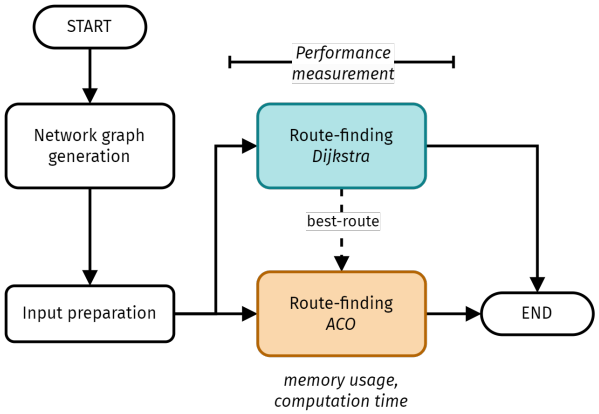


Fig. 4. Measurement flow

B. Implementation

The implementation of the system was started by adding a user interface (UI) to the website to make it easier for users to find public transit routes. The system—by default—will automatically present the user's location as the default position on the map. Web technologies, e.g., HTML5, Bootstrap, Javascript, and PHP programming languages, are used to develop the system. Users merely need to enter or choose their starting and finishing locations, click the process button, and the results from the Dijkstra and ACO algorithms will then be shown on the screen.

Fig. 5 shows the process of how the graph data was created and how the graph is being used as the working environment of the study. The system is retrieving all of the line data that has been saved in the database, which is the first stage in producing the graph data. Then, a list of each line's interchange points is compiled from the points from each line that were stored in the database. The information was then used to develop the public transit network graph data, which was subsequently processed using the Dijkstra and ACO algorithms.

The Dijkstra method is used to process the graph data after it has been created, and it yields one optimized route. Then, the ACO tried to generate a route that agreed with the yielded route. The performance comparison of the ACO and Dijkstra methods was measured during execution, taking into account memory usage and computation performance. This is to see whether ACO's time and memory requirements for the computation process were better, worse, or similar to those of Dijkstra.

C. Measurement

Both methods were tested by selecting several start and end locations that represent the near, medium, and far route categories. A route is said to be near when its air distance is less than 5 kilometers, whereas the distance of a route is said to be medium when its air distance is between 5 and 10 kilometers. Any route whose air distance is more than 10 kilometers falls into the far category.

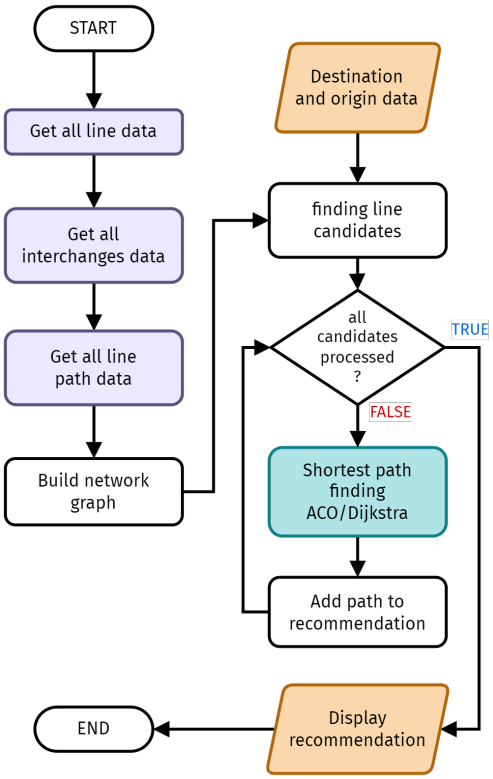


Fig. 5. Architecture System

For the purpose of gathering data, the testing is carried out 30 times for each set of start and end points that correspond to the far and near route categories. Such strategies guaranteed that numerous trials and possibilities were carried out for each circumstance, reducing bias or inaccurate results.

IV. Result and Discussion

A. Measurement Data

During the testing process, each pair of starting and ending points is repeated 30 times for each categorization, resulting in 360 measurement data. The performance impact of Dijkstra and ACO algorithms on the specified scenarios was addressed in detail. Each pair undergoes 30 trials, ensuring the validity and consistency of the results. This method additionally takes into account arbitrary variations that could happen during a single measurement or as a result of JavaScript's automatic garbage collection process. The location—latitude-longitude—inputs, which were used as input route pairings, are shown in Table I.

The "Category" from Table I has been simplified in Tables II and III. Specifically, "near 1," "near 2," "medium 1," "medium 2," and "far 1," "far 2," becoming "Far." The results of memory consumption from the approaches (ACO and Dijkstra) with a high number of data points (n) are shown in Table II. The standard deviation (sd), minimum, mean, and maximum are displayed, suggesting varied performances among these categories.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on August 27,2026 at 09:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ROUTE PAIR DATA

Category	Source	Destination	Distance (km)
near 1	S1	D1	4,83
near 2	S2	D2	1,77
medium 1	S3	D4	5,63
medium 2	S4	D4	5,03
far 1	S5	D5	13,69
far 2	S6	D6	13,57

TABLE II
MEMORY USAGE DATA (BYTES)

Method	Distance	n	min	mean	max	sd
ACO	Near	58	180	41600	84322	24697
ACO	Medium	58	254	62352	514548	83722
ACO	Far	58	1116	33823	97495	26569
Dijkstra	Near	58	132	44819	92715	26206
Dijkstra	Medium	58	6072	44113	84131	17129
Dijkstra	Far	58	1156	44118	99440	29875

The time performance procedure based on sample size, distance grouping, and statistics such as minimum, mean, maximum, and standard deviation (sd) in milliseconds is shown in Table III. The results show that different distance categories have varied processing times for ACO and Dijkstra. All things considered, the tables provide information on the amount of memory used and the processing time of the Dijkstra and ACO algorithms for route optimization.

B. Analysis

The Shapiro-Wilk tests' p-values for the time and memory data all fell below 0.05, indicating that their distributions are not normal. Thus, further analyses will be carried out using non-parametric approaches. A graphical depiction of the data list for the utilized RAM is shown in Fig. 6. This graph depicts the performance data comparison in terms of memory usage during the computation between ACO and Dijkstra, while Fig. 7 illustrates the comparison of the processing time between both methods.

Based on the analysis of Figure 6 and Figure 7, it can be concluded that the Ant-Colony Optimization (ACO) algorithm performs exceptionally well and can rival or even surpass the results of the Dijkstra algorithm in the context of memory usage and processing time. Results from statistical studies show that ACO and Dijkstra show identical efficiency in terms of memory usage, with no discernible difference in memory utilization between the two across different trip circumstances.

On the other hand, when processing time performance is taken into account, strong evidence surfaces. In some cases, especially for long-distance travel routes, ACO performs better in terms of processing time than Dijkstra. The data shown in Table III, which indicates shorter ACO processing times than Dijkstra processing durations, makes this very clear.

TABLE III
PROCESSING TIME DATA (MS)

Method	Distance	n	min	mean	max	sd
ACO	Near	60	1.3	49.6	241	58.5
ACO	Medium	60	2	52.3	198	49
ACO	Far	60	2	29.8	118	29.7
Dijkstra	Near	60	3	53.5	297	58.3
Dijkstra	Medium	60	3.8	54	260	51.6
Dijkstra	Far	60	3.5	60.9	289	61.5

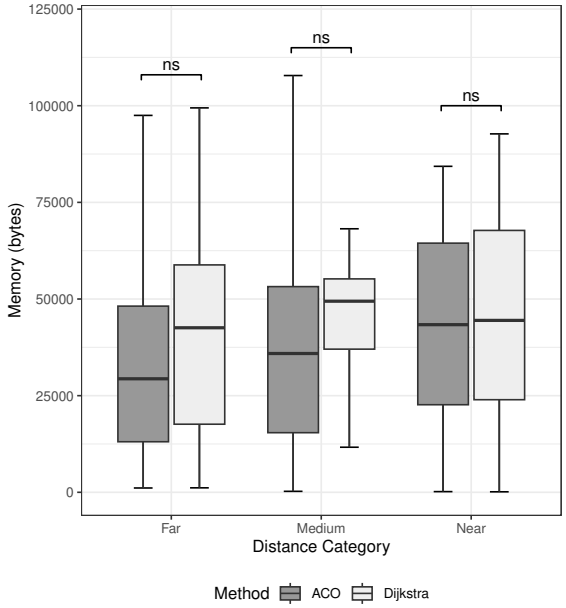


Fig. 6. Memory usage of ACO and Dijkstra
ns: not significant

To find out what is causing the variations in processing time performance, further thorough investigation can be done. The Greedy technique taken by Dijkstra, which could take longer to process all the data even after determining the optimal solution, is one element that could be very important. As a result, the study's findings offer insightful information about the relative effectiveness of ACO and Dijkstra in relation to resource usage and processing time.

V. LIMITATION

There are several constraints established that limit the scope and context of this study. First, the route data is obtained from the public transportation network in Malang City. Second, the testing phase was focused on assessing memory usage and processing time as the key performance metrics of this study and was measured on a smartphone of a particular type, which may yield different measurement results when carried out on different devices.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on August 27,2026 at 09:44:20 UTC from IEEE Xplore. Restrictions apply.

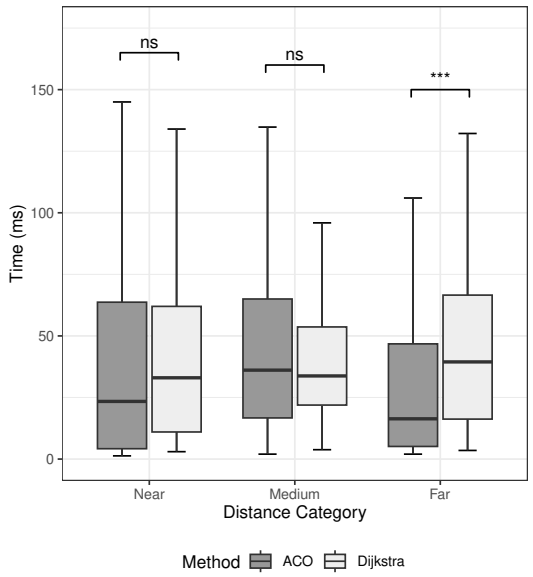


Fig. 7. Processing time comparison between ACO and Dijkstra
ns: not significant

VI. Conclusion

Overall, the analysis's findings demonstrate that for every studied trip route scenario, ACO and Dijkstra perform about equally in terms of memory utilization. What's intriguing, though, is that there are compelling signs that ACO can make recommendations more quickly than Dijkstra—especially for lengthy travel routes. Consequently, it can be concluded that ACO is a competitive alternative to solve routing difficulties, and in certain situations, it may even be able to deliver faster solutions without compromising memory consumption efficiency. An automated system that can alternate between the ACO and Dijkstra methods depending on the properties of the supplied data could be explored as a future development idea. Dijkstra, for example, might be a more memory-efficient method for datasets with a lower volume. On the other hand, turning on ACO may be able to maximize processing speed for bigger datasets. Therefore, the creation of such an automated process could improve the route algorithm's adaptability and efficiency, guaranteeing that the best approach is used in each unique circumstance.

References

[1] J. Ning, Q. Zhang, C. Zhang, and B. Zhang, "A best-path-updating information-guided ant colony optimization algorithm," *Inf. Sci.*, vol. 433-434, pp. 142–162, 2018.

[2] R. Abduljabbar, H. Dia, S. Liyanage, and S. A. Bagloee, "Applications of artificial intelligence in transport: An overview," *Sustainability*, vol. 11, no. 1, 2019. [Online]. Available: https://www.mdpi.com/2071-1050/11/1/189

[3] Z. Nie and H. Zhao, "Research on robot path planning based on dijkstra and ant colony optimization," *2019 International Conference on Intelligent Informatics and Biomedical Sciences (ICIIBMS)*, pp. 222–226, 2019.

[4] A. Pinandito and C. Wulandari, "Integrating douglas-peucker line simplification into routeboxer algorithm on a map-based android application," pp. 213–219, 11 2020.

[5] D. Baeza, C. F. Ihle, and J. M. Ortiz, "A comparison between aco and dijkstra algorithms for optimal ore concentrate pipeline routing," *Journal of Cleaner Production*, vol. 144, pp. 149–160, 2017.

[6] P. Sharan, "Comparison of dijkstra algorithm with ant colony optimization algorithm using random topology in all-optical network using rwa," *International Journal of Research in Engineering and Technology*, vol. 04, pp. 66–69, 2015. [Online]. Available: https://api.semanticscholar.org/CorpusID:2809596

[7] A. A. Shojaie and S. E. Seyedi Bariran, "A comparison of extended dijkstra and aco algorithm for shortest path problem in dynamic networks with delay times," *Advances in Industrial Engineering*, vol. 55, no. 1, pp. 1–26, 2021.

[8] P. Aries, S. Zen, Y. Mike, and W. Neny, "Perbandingan metode ant colony optimization dan dijkstra untuk pengembangan sistem pengiriman barang di kantor pos area surabaya timur berbasis j2me," *Emitter*, 2010. [Online]. Available: http://repo.pens.ac.id/205/

[9] N. Makariye, "Towards shortest path computation using dijkstra algorithm," *2017 International Conference on IoT and Application (ICIOT)*, pp. 1–3, 2017.

[10] E. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, pp. 269–271, 1959.

[11] P. N. Ky Phuc and N. L. Phuong Thao, "Ant colony optimization for multiple pickup and multiple delivery vehicle routing problem with time window and heterogeneous fleets," *Logistics*, vol. 5, no. 2, 2021. [Online]. Available: https://www.mdpi.com/2305-6290/5/2/28

[12] X. Liu, H. Yi, and Z. Ni, "Application of ant colony optimization algorithm in process planning optimization," *Journal of Intelligent Manufacturing*, vol. 24, pp. 1–13, 2013.

[13] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction To Algorithms*, ser. Mit Electrical Engineering and Computer Science. MIT Press, 2001. [Online]. Available: https://books.google.co.id/books?id=NLngYyWFl_YC

[14] A. Bozyigit, G. Alankus, and E. Nasiboğlu, "Public transport route planning: Modified dijkstra's algorithm," *2017 International Conference on Computer Science and Engineering (UBMK)*, pp. 502–505, 2017.

[15] P. Tirastittam and P. Waiyawuththanapoom, "Public transport planning system by dijkstra algorithm: Case study bangkok metropolitan area," *World Academy of Science, Engineering and Technology, International Journal of Computer and Information Engineering*, vol. 1, 2014.

[16] Y. Martynova, Y. Martynov, D. Mustafina, and V. V. Asmolovskiy, "Ant colony algorithm for rational transit network design of urban passenger transport," *2014 International Conference on Mechanical Engineering, Automation and Control Systems (MEACS)*, pp. 1–5, 2014.

[17] L. Xie, M. Merschformann, N. Kliewer, and L. Suhl, "Metaheuristics approach for solving personalized crew rostering problem in public bus transit," *Journal of Heuristics*, vol. 23, pp. 321–347, 2017.

[18] J. Euchi and R. Mraihi, "The urban bus routing problem in the tunisian case by the hybrid artificial ant colony algorithm," *Swarm Evol. Comput.*, vol. 2, pp. 15–24, 2012.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on August 27,2026 at 09:44:20 UTC from IEEE Xplore. Restrictions apply.